

01. Frontend - Học Và Nhớ Library74 Từ Gốc

Tài liệu này xem bạn là người mới học lập trình web ở mức cơ bản. Mục tiêu không phải học thuộc code, mà là hiểu frontend đang giải quyết vấn đề gì, vì sao chọn cách đó, và khi hội đồng hỏi thì bạn giải thích được bằng logic.

Lộ trình học toàn project:

1. FE - Frontend: người dùng nhìn thấy gì, bấm gì, dữ liệu đi đâu.
2. DB - Database: dữ liệu được lưu như thế nào.
3. BE - Backend: nghiệp vụ, bảo mật, API.
4. AI - Recommendation, semantic search, xử lý tri thức.
5. Deploy/DevOps - đưa hệ thống chạy thật trên server.

Trong file này ta học phần FE trước.

1. Frontend Là Gì?

1.1. Vấn Đề Thực Tế

Người dùng không tương tác trực tiếp với database hay backend. Họ cần một giao diện để:

- Tìm kiếm sách.
- Đăng nhập, đăng ký.
- Mượn sách, xem sách đang mượn.
- Đặt trước sách.
- Xem phí phạt.
- Thủ thư quản lý sách, bản sao, giao dịch.
- Admin quản lý tài khoản.

Frontend chính là phần đứng giữa người dùng và hệ thống.

Nói đơn giản:

```
Người dùng -> giao diện web -> gọi API backend -> backend xử lý -> trả dữ liệu -> giao diện hiển thị
```

1.2. Định Nghĩa

Frontend là phần giao diện chạy trên trình duyệt, gồm HTML, CSS, JavaScript và các framework hỗ trợ. Frontend chịu trách nhiệm hiển thị dữ liệu, nhận thao tác người dùng, gọi API, xử lý trạng thái giao diện.

Trong project Library74, frontend nằm ở:

LMS_FE/

2. Công Nghệ Frontend Của Project

Project dùng các công nghệ chính:

Công nghệ	Vai trò
React	Xây dựng giao diện bằng component
TypeScript	JavaScript có kiểu dữ liệu rõ ràng hơn
Vite	Công cụ chạy dev server và build frontend
React Router	Điều hướng giữa các trang
Axios	Gọi API backend
Tailwind CSS	Viết CSS bằng utility class
Context API	Quản lý trạng thái dùng chung
Jest	Test frontend
Nginx	Serve frontend khi deploy production

3. React Là Gì?

3.1. Vấn Đề Thực Tế

Nếu viết web truyền thống bằng HTML/CSS/JS thuần, khi giao diện lớn lên sẽ rất khó quản lý:

- Header lặp lại nhiều nơi.
- Sidebar của user, librarian, admin khác nhau.
- Trang nào cũng phải tự xử lý loading, dữ liệu, lỗi.
- Khi dữ liệu thay đổi, phải tự tìm DOM rồi sửa thủ công.

Ví dụ: người dùng đăng nhập xong, nút "Đăng nhập" phải đổi thành tên người dùng. Nếu dùng JavaScript thuần, ta phải tự tìm element và sửa text. Khi app lớn, cách này dễ rối.

3.2. Định Nghĩa

React là thư viện JavaScript để xây dựng giao diện theo mô hình component.

Component là một khối giao diện độc lập, có thể tái sử dụng.

Ví dụ trong Library74:

```
components/BrandLogo.tsx
components/ThemeModeToggle.tsx
```

```
components/LanguageSwitcher.tsx
components/user_pages/Sidebar.tsx
components/admin_pages/AdminLayout.tsx
```

Mỗi component giống một "viên gạch" tạo nên giao diện.

3.3. Đặc Trưng Của React

React có 3 đặc trưng quan trọng:

1. Component-based: chia UI thành nhiều component nhỏ.
2. Declarative UI: mô tả giao diện nên trông như thế nào theo state.
3. State-driven rendering: state đổi thì UI tự render lại.

3.4. State Là Gì?

State là dữ liệu tạm thời của giao diện.

Ví dụ:

- Người dùng đang nhập gì vào ô search.
- Modal đang mở hay đóng.
- Sidebar đang thu gọn hay mở rộng.
- Danh sách sách mới đã load xong chưa.

Trong React, state thường được tạo bằng `useState`.

Ví dụ logic:

```
const [searchInput, setSearchInput] = useState('');
```

Ý nghĩa:

- `searchInput`: giá trị hiện tại.
- `setSearchInput`: hàm dùng để cập nhật giá trị.
- `''`: giá trị ban đầu là chuỗi rỗng.

3.5. Effect Là Gì?

Effect là hành động phụ xảy ra sau khi component render, ví dụ gọi API, đọc localStorage, đăng ký sự kiện.

Trong React, effect thường dùng `useEffect`.

Ví dụ thực tế trong trang chủ:

```
Khi trang HomePage mở lên -> gọi API lấy sách mới, sách mượn nhiều,
category, stats, review.
```

Đây là effect vì nó không chỉ vẽ UI, mà còn làm việc với thế giới bên ngoài.

4. TypeScript Là Gì?

4.1. Vấn Đề Thực Tế

JavaScript rất linh hoạt nhưng dễ sai kiểu dữ liệu.

Ví dụ backend trả:

```
{
  "totalPublications": 32,
  "activeUsers": 81
}
```

Nếu frontend nhầm `totalPublications` là chuỗi hoặc viết sai tên thành `totalPublication`, lỗi có thể chỉ xuất hiện khi chạy web.

4.2. Định Nghĩa

TypeScript là JavaScript có bổ sung hệ thống kiểu dữ liệu. File TypeScript thường có đuôi:

```
.ts
.tsx
```

Trong project:

- `.ts`: logic không chứa JSX.
- `.tsx`: component React có JSX.

4.3. JSX Là Gì?

JSX là cú pháp viết giao diện giống HTML bên trong JavaScript/TypeScript.

Ví dụ:

```
return <h1>Library74</h1>;
```

Nó không phải HTML thuần. Vite/React sẽ biên dịch JSX thành JavaScript để trình duyệt hiểu.

4.4. Interface/Type Là Gì?

Interface hoặc **type** mô tả hình dạng dữ liệu.

Ví dụ trong project có kiểu `PublicLibraryStats`:

```
totalPublications: number
activeUsers: number
totalBorrows: number
```

Khi component dùng `stats.totalPublications`, TypeScript giúp kiểm tra `stats` có field đó hay không.

Lưu ý: TypeScript giúp giảm lỗi, nhưng không thay thế việc test. Nếu truyền thiếu props hoặc logic runtime sai, vẫn có thể crash.

5. Vite Là Gì?

5.1. Vấn Đề Thực Tế

Code React/TypeScript không chạy trực tiếp y nguyên trên trình duyệt production. Ta cần công cụ để:

- Chạy dev server khi lập trình.
- Biên dịch TypeScript/JSX.
- Gộp và tối ưu file.
- Tạo thư mục `dist` để deploy.

5.2. Định Nghĩa

Vite là build tool cho frontend hiện đại. Nó giúp chạy app nhanh khi dev và build app tối ưu khi deploy.

Các lệnh trong `LMS_FE/package.json`:

```
npm run dev
npm run build
npm run preview
npm run test
```

Ý nghĩa:

- `npm run dev`: chạy local để code.
- `npm run build`: build production, tạo thư mục `dist`.
- `npm run preview`: xem thử bản production build ở local.
- `npm run test`: chạy test frontend.

5.3. Quy Trình Làm Việc Đúng

Khi sửa FE:

```
cd LMS_FE
npm run dev
```

Sau khi code xong:

```
npm run build  
npm run test
```

Khi ổn mới deploy production.

Đây là cách đúng để tránh tình trạng "push lên production rồi mới biết lỗi".

6. Routing Là Gì?

6.1. Vấn Đề Thực Tế

Một app thư viện có nhiều màn hình:

- /publicpage
- /publicpage/search
- /userpage/dashboard
- /userpage/my-books
- /librarianpage/books
- /adminpage/users

Người dùng cần chuyển trang mà không reload toàn bộ website.

6.2. Định Nghĩa

Routing là cơ chế ánh xạ URL với component.

Trong project dùng:

```
react-router-dom
```

Trong **App.tsx**, các route được chia theo nhóm:

```
PUBLIC_PREFIX = /publicpage  
USER_PREFIX = /userpage  
LIB_PREFIX = /librarianpage  
ADMIN_PREFIX = /adminpage
```

6.3. HashRouter Là Gì?

Project dùng:

HashRouter

URL sẽ có dạng:

```
https://library74.uk/#/publicpage
```

Phần sau dấu # do frontend xử lý. Cách này giúp deploy SPA dễ hơn vì server chỉ cần trả `index.html`, còn route chi tiết do React Router quyết định.

6.4. Protected Route Là Gì?

Protected route là route cần đăng nhập hoặc cần đúng quyền.

Ví dụ:

- Student mới vào được `/userpage/dashboard`.
- Librarian mới vào được `/librarianpage/circulation`.
- Admin mới vào được `/adminpage/users`.

Trong `App.tsx`, component `ProtectedRoute` kiểm tra:

```
Nếu chưa đăng nhập -> chuyển về login  
Nếu sai role -> chuyển về dashboard phù hợp  
Nếu đúng role -> cho xem trang
```

7. API Là Gì?

7.1. Vấn Đề Thực Tế

Frontend không tự có dữ liệu sách, user, lượt mượn. Dữ liệu nằm ở backend/database.

Frontend phải hỏi backend:

```
"Cho tôi danh sách sách mới"  
"Cho tôi thông tin user đang đăng nhập"  
"Cho tôi danh sách phí phạt"
```

7.2. Định Nghĩa

API là giao diện để các phần mềm nói chuyện với nhau.

Trong web, API thường là HTTP endpoint.

Ví dụ:

```
GET /api/v1/publications/public-stats
GET /api/v1/users/me
POST /api/v1/auth/login
```

7.3. Axios Là Gì?

Axios là thư viện JavaScript dùng để gọi HTTP API.

Trong project, axios được cấu hình ở:

```
LMS_FE/api/axiosInstance.ts
```

File này tạo một axios instance dùng chung cho toàn app.

7.4. Base URL Là Gì?

Base URL là địa chỉ gốc của API.

Trong code:

```
baseUrl: import.meta.env.VITE_API_BASE_URL || "http://localhost:3000/api"
```

Nghĩa là:

- Nếu có biến môi trường `VITE_API_BASE_URL`, dùng biến đó.
- Nếu không có, dùng default local.

7.5. Interceptor Là Gì?

Interceptor là hàm chặn request/response để xử lý chung.

Trong project có 2 loại:

1. Request interceptor.
2. Response interceptor.

Request interceptor làm các việc:

- Lấy `accessToken` từ `localStorage`.
- Gắn vào header `Authorization`.
- Gắn ngôn ngữ vào `Accept-Language`.

Response interceptor làm các việc:

- Unwrap `response.data`.
- Nếu token hết hạn thì gọi refresh token.
- Nếu refresh thất bại thì xóa token và đưa user về login.

Đây là một điểm rất quan trọng khi bảo vệ, vì nó thể hiện frontend có xử lý bảo mật phiên đăng nhập.

8. Token, localStorage Và Đăng Nhập

8.1. Token Là Gì?

Token là chuỗi ký tự đại diện cho phiên đăng nhập.

Sau khi login thành công, backend trả về:

- **accessToken**: dùng để gọi API.
- **refreshToken**: dùng để xin access token mới khi access token hết hạn.

8.2. localStorage Là Gì?

localStorage là bộ nhớ nhỏ trong trình duyệt. Dữ liệu vẫn còn sau khi reload tab.

Project lưu:

```
accessToken  
refreshToken  
userType  
library74.language  
library74.theme
```

8.3. Luồng Đăng Nhập

```
User nhập email/password  
-> frontend gọi API login  
-> backend kiểm tra  
-> backend trả token  
-> frontend lưu token  
-> user vào dashboard  
-> các request sau tự gắn Authorization: Bearer <token>
```

8.4. Luồng Refresh Token

```
Frontend gọi API  
-> backend báo access token hết hạn  
-> frontend dùng refresh token xin access token mới  
-> nếu thành công: lưu token mới và gọi lại request cũ  
-> nếu thất bại: xóa token, chuyển về login
```

Điểm hay:

- Người dùng không bị logout ngay khi access token hết hạn.

- Nhiều request cùng hết hạn không gọi refresh ồ ạt, vì code có `failedQueue`.

9. Context API Là Gì?

9.1. Vấn Đề Thực Tế

Nhiều component cần dùng chung dữ liệu:

- User đang đăng nhập chưa?
- Role là student/librarian/admin?
- Ngôn ngữ hiện tại là VI hay EN?
- Theme sáng/tối?
- Notification realtime?

Nếu truyền dữ liệu qua props từng tầng, code rất rối.

9.2. Định Nghĩa

Context API là cơ chế của React để chia sẻ dữ liệu cho nhiều component mà không phải truyền props qua từng cấp.

Trong project có:

```
contexts/AuthContext.tsx
contexts/LanguageContext.tsx
contexts/ThemeContext.tsx
contexts/NotificationContext.tsx
contexts/UploadContext.tsx
contexts/AppDialogContext.tsx
```

9.3. AuthContext

Quản lý trạng thái đăng nhập:

- User type.
- Login/logout.
- Kiểm tra role.

9.4. LanguageContext

Quản lý đa ngôn ngữ:

- Tiếng Việt.
- Tiếng Anh.
- Hàm `t('key')` để lấy nội dung.

Ví dụ:

```
t('nav.home')
```

Thay vì hard-code "**Trang chủ**" ở mọi nơi, ta dùng key. Khi đổi ngôn ngữ, UI đổi theo.

9.5. ThemeContext

Quản lý sáng/tối/system.

Theme được lưu trong:

```
localStorage key: library74.theme
```

10. Tailwind CSS Là Gì?

10.1. Vấn Đề Thực Tế

CSS truyền thống dễ bị:

- Trùng tên class.
- File CSS quá dài.
- Khó biết style nào đang ảnh hưởng component nào.

10.2. Định Nghĩa

Tailwind CSS là framework CSS theo kiểu utility-first.

Thay vì viết:

```
.button {  
  background: blue;  
  color: white;  
  padding: 8px 12px;  
}
```

Ta viết trực tiếp class:

```
<button className="bg-blue-600 text-white px-3 py-2 rounded-lg">
```

10.3. Ưu Điểm

- Viết UI nhanh.
- Ít phải đặt tên class.
- Nhìn component là thấy style.

- Dễ responsive bằng `sm:`, `md:`, `lg:`.

10.4. Nhược Điểm

- Class dài.
- Người mới nhìn có thể thấy rối.
- Cần thống nhất style để tránh UI mỗi nơi một kiểu.

Trong project, Tailwind được dùng gần như toàn bộ UI.

11. Component Layout Theo Role

11.1. Vấn Đề Thực Tế

Student, librarian, admin có giao diện khác nhau:

- Student cần dashboard, sách đang mượn, phí phạt.
- Librarian cần quản lý mượn trả, sách, bản sao.
- Admin cần quản lý user, librarian, chính sách, audit log.

Nếu dùng một layout chung cho tất cả, code sẽ khó bảo trì.

11.2. Cách Project Giải Quyết

Project chia layout:

```
components/public_pages/Layout.tsx
components/user_pages/Sidebar.tsx
components/librarian_pages/LibrarianLayout.tsx
components/admin_pages/AdminLayout.tsx
```

Tư duy:

```
Public layout -> khách chưa đăng nhập
User layout -> sinh viên
Librarian layout -> thủ thư
Admin layout -> quản trị
```

Mỗi layout quản lý:

- Header/sidebar.
 - Menu.
 - Logout.
 - Điều hướng.
 - Vùng nội dung chính.
-

12. Pages Và Services

12.1. Page Là Gì?

Page là component đại diện cho một màn hình lớn.

Ví dụ:

```
pages/public_pages/HomePage.tsx
pages/user_pages/MyBooksPage.tsx
pages/librarian_pages/Circulation.tsx
pages/admin_pages/AdminUsersPage.tsx
```

12.2. Service Là Gì?

Service là file gom các hàm gọi API theo nghiệp vụ.

Ví dụ:

```
api/publicationsService.ts
api/usersService.ts
api/adminService.ts
api/transactionsService.ts
api/fineService.ts
```

Tại sao cần service?

Nếu gọi axios trực tiếp trong mọi page, code bị lặp và khó sửa. Khi backend đổi endpoint, phải sửa nhiều nơi.

Cách tốt hơn:

```
Page -> gọi service -> service gọi axios -> backend
```

Ví dụ luồng trang chủ:

```
HomePage
-> publicationsService.getPublicStats()
-> axiosInstance.get('/publications/public-stats')
-> backend trả totalPublications, activeUsers...
-> HomePage setStats
-> UI render số liệu
```

13. Loading, Error Và Empty State

13.1. Vấn Đề Thực Tế

API không trả kết quả ngay lập tức. Có 3 trạng thái thường gặp:

1. Đang tải.
2. Tải thành công.
3. Tải lỗi hoặc không có dữ liệu.

13.2. Loading State

Loading state là trạng thái đang chờ dữ liệu.

Ví dụ:

```
loadingNewest = true
```

Khi API trả xong:

```
loadingNewest = false
```

13.3. Empty State

Empty state là giao diện khi không có dữ liệu.

Ví dụ:

```
Không có sách mới nào.  
Chưa có đánh giá hệ thống nào.
```

13.4. Error State

Error state là trạng thái lỗi, ví dụ backend không phản hồi, token hết hạn, API trả lỗi.

Frontend tốt không được crash trắng/đen màn hình. Nó cần:

- Hiện thị lỗi dễ hiểu.
- Cho user thử lại.
- Ghi log khi dev.

14. Sự Cố Màn Hình Đen Vừa Gặp

14.1. Hiện Tượng

Khi bấm `/publicpage`, trình duyệt đen/trắng và không render nội dung.

14.2. Nguyên Nhân Logic

Frontend bị crash runtime.

Một component dùng biến `stats`, nhưng biến đó không tồn tại trong phạm vi component.

Logic sai:

```
HeroSection dùng stats.totalPublications
nhưng HeroSection không nhận stats từ HomePage
```

14.3. Cách Sửa

Truyền `stats` từ component cha xuống component con bằng props.

```
HomePage có stats
-> <HeroSection stats={stats} />
-> HeroSection nhận props stats
-> dùng stats.totalPublications an toàn
```

14.4. Bài Học

Khi sửa frontend:

1. Chạy local.
2. Build production.
3. Preview production build.
4. Test bằng trình duyệt.
5. Rồi mới deploy.

Không nên sửa xong deploy ngay nếu chưa build/test.

15. Production Build Và Deploy FE

15.1. Dev Khác Production Như Thế Nào?

Khi dev:

```
npm run dev
```

Vite phục vụ code nhanh, hỗ trợ hot reload.

Khi production:

```
npm run build
```

Vite tạo file tối ưu trong **dist**.

Production không chạy dev server. Production dùng Nginx để serve file tĩnh.

15.2. Dockerfile FE

Frontend có:

```
LMS_FE/Dockerfile
```

Ý tưởng:

```
Stage 1: dùng Node build React app  
Stage 2: dùng Nginx serve dist
```

Đây gọi là **multi-stage build**.

Ưu điểm:

- Image production nhẹ hơn.
- Không cần mang source code/node_modules vào container chạy thật.
- Chỉ serve file tĩnh đã build.

15.3. Nginx Là Gì?

Nginx là web server. Trong frontend, Nginx chủ yếu làm nhiệm vụ trả file:

- **index.html**
- **.js**
- **.css**
- ảnh/logo

Với SPA, nếu user vào route bất kỳ, Nginx nên trả **index.html**, rồi React Router xử lý route.

16. Test Frontend

16.1. Vấn Đề Thực Tế

Frontend dễ lỗi do:

- Đổi UI làm mất button.
- Đổi API làm sai dữ liệu.
- Đổi context làm crash component.

- Đổi language/theme làm test fail.

16.2. Jest Là Gì?

Jest là framework test JavaScript/TypeScript.

Project có test trong:

```
LMS_FE/src/**/*.test.ts  
LMS_FE/src/**/*.test.tsx
```

Chạy:

```
cd LMS_FE  
npm run test
```

16.3. Testing Library Là Gì?

Testing Library giúp test component theo cách người dùng nhìn thấy và thao tác, thay vì test implementation chi tiết.

Ví dụ nên test:

- Có text "Đăng nhập" không?
- Bấm button có gọi đúng hàm không?
- Khi API lỗi có hiện message không?

Không nên quá phụ thuộc vào class CSS nội bộ.

17. Cách Trình Bày FE Trước Hội Đồng

Bạn có thể trình bày theo khung sau:

17.1. Mở Đầu

"Frontend của hệ thống Library74 là một SPA được xây dựng bằng React, TypeScript và Vite. Mục tiêu của frontend là cung cấp giao diện cho ba nhóm người dùng: sinh viên, thủ thư và admin."

17.2. Kiến Trúc FE

"Frontend được tổ chức theo route và role. Public pages dành cho khách, user pages cho sinh viên, librarian pages cho thủ thư, admin pages cho quản trị viên. Các layout riêng giúp phân tách giao diện và quyền truy cập."

17.3. Giao Tiếp Backend

"Frontend không truy cập database trực tiếp mà gọi backend thông qua API. Các API được gom trong thư mục `api`, sử dụng axios instance dùng chung. Request interceptor tự động gắn token và ngôn ngữ; response interceptor xử lý refresh token khi phiên đăng nhập hết hạn."

17.4. Bảo Mật FE

"Frontend có protected route để ngăn user truy cập sai vai trò. Tuy nhiên bảo mật thật sự vẫn nằm ở backend; frontend chỉ là lớp trải nghiệm và điều hướng."

17.5. Build/Deploy

"Khi deploy, frontend được build bằng Vite thành static files trong `dist`, sau đó Nginx serve các file này trong Docker container. Caddy đứng ngoài làm reverse proxy và HTTPS."

18. Các Câu Hỏi Đồng Có Thể Hỏi

Câu 1: Vì sao dùng React?

Vì hệ thống có nhiều giao diện phức tạp, nhiều state và nhiều role. React giúp chia UI thành component tái sử dụng, dễ bảo trì hơn so với HTML/JS thuần.

Câu 2: Vì sao dùng TypeScript?

TypeScript giúp kiểm tra kiểu dữ liệu sớm, giảm lỗi do sai field API hoặc truyền props sai. Với hệ thống lớn, type giúp code dễ đọc và dễ refactor hơn.

Câu 3: Frontend có bảo mật không?

Frontend có protected route, lưu token và tự gắn Authorization header. Nhưng frontend không phải lớp bảo mật cuối cùng vì người dùng có thể sửa code trên trình duyệt. Backend vẫn phải kiểm tra token và role ở mọi API quan trọng.

Câu 4: Khi token hết hạn thì sao?

Response interceptor kiểm tra lỗi token hết hạn. Nếu có refresh token, frontend gọi API refresh để lấy access token mới, lưu lại và retry request cũ. Nếu refresh thất bại thì xóa token và đưa user về login.

Câu 5: Vì sao dùng HashRouter?

HashRouter giúp SPA hoạt động ổn khi deploy static file, vì phần route sau dấu `#` do frontend xử lý. Server chỉ cần trả `index.html`.

Câu 6: Làm sao tránh lỗi màn hình trắng/đen?

Quy trình đúng là chạy local, build production, preview production build, kiểm tra console/browser trước khi deploy. Ngoài ra cần kiểm soát loading/error state và không dùng biến ngoài phạm vi component.

19. Checklist Học FE

Bạn nên học theo thứ tự:

1. Hiểu SPA, frontend/backend/database khác nhau thế nào.
2. Hiểu React component, props, state, effect.
3. Hiểu TypeScript, interface/type, JSX.
4. Hiểu routing và protected route.
5. Hiểu axios, API service, request/response interceptor.
6. Hiểu auth token, refresh token, localStorage.
7. Hiểu context: auth, theme, language, notification.
8. Hiểu layout theo role.
9. Hiểu build bằng Vite và deploy bằng Nginx/Docker.
10. Biết demo một luồng thực tế: search sách, login, mượn/trả, admin sửa user.

20. Bài Tập Tự Kiểm Tra

Bài 1

Giải thích bằng lời luồng sau:

```
User vào /publicpage -> search sách -> frontend gọi API -> backend trả kết quả -> frontend render danh sách sách
```

Bài 2

Tìm trong code:

```
api/publicationsService.ts  
pages/public_pages/SearchPage.tsx
```

Trả lời:

- Page gọi hàm service nào?
- Service gọi endpoint nào?
- Dữ liệu trả về được lưu vào state nào?

Bài 3

Tìm trong code:

```
contexts/AuthContext.tsx  
api/axiosInstance.ts  
App.tsx
```

Trả lời:

- Token lưu ở đâu?
- Request gắn token ở đâu?
- Route kiểm tra role ở đâu?

Bài 4

Tự trình bày trong 3 phút:

"Frontend Library74 được tổ chức như thế nào và giao tiếp với backend ra sao?"

Nếu nói trôi chảy câu này, bạn đã nắm được nền FE ở mức bảo vệ được.